

Course: Software Engineering

Course Code: 63301

פרויקט גמר

תאור כללי

פרויקט הגמר מהווה סיכום של החומר הנלמד בקורס. על הפרויקט לשקף את היכולת לפתח מערכת תוכנה באופן הנדסי נכון. השיקוף אמור לכלול הן את אופן ביצוע הפרויקט (כלומר את השלבים שהפרויקט עבר במהלך הפיתוח) והן את החומרים שמהווים את התוצר באופן הנדסי (כלומר כתיבה תוך שימוש בכלים ראויים כגון שימוש בעצמים באופן ראוי, שימוש בתבניות תיכון (design Pattern), עמידות מערכת ונוחות למשתמש.

מטרות הפרויקט

- הכנת עבודה עצמאית המסכמת ומיישמת את הידע שנרכש בתקופת הלימודים.
- העבודה תכלול: הגדרת דרישות, ותכנון של המערכת.
- לימוד השימוש בספרות טכנית.
- הכרה ותרגול של תהליכי עבודה מסודרים.

דגשים כלליים

- הפרויקט יבוצע בקבוצות של לכל הפחות שניים ולכל היותר ארבעה סטודנטים.
- נושא הפרויקט יבחר מתוך הרשימה המופיעה למטה.

נושאים שיבדקו

1. קיום מסמכי דרישות.
2. קיום מסמכי עיצוב וארכיטקטורה ושימוש בכלים שנלמדו.

אפשרויות לנושא הפרויקט

כל המערכות צריכות לשמור מידע מהמשתמש ולהיות אינטראקטיביות (כלומר לאפשר עבודה מול המערכת באמצעות ממשק משתמש)

1. Hotel Reservation System

- **Scenario:** Guests search for available rooms, make reservations, check in, and check out. The system manages room availability and prevents overlapping reservations.
- **Skills Practiced:** Implement Hotel, Room, Guest, and Reservation classes, detect date conflicts, calculate stay costs, generate occupancy reports.
- **Key Concepts:** OOP (aggregation and composition), Collections, date handling, Strategy pattern (pricing policies) or Factory pattern (room creation).

2. Student Course Registration System

- **Scenario:** Students enroll in available courses considering capacity, time conflicts, and prerequisites. The system should validate enrollment rules and allow students to view or drop courses.
- **Skills Practiced:** Implement Course and Student classes, check for time conflicts using schedule data structures, maintain course rosters with capacity tracking, design a text-based menu system.
- **Key Concepts:** OOP principles (encapsulation, association), input validation, exception handling, object collections.

3. Quiz and Grading System

- **Scenario:** Teachers design quizzes with different question types: Multiple Choice Questions (MCQ) - True/False. Students take quizzes, receive instant scores, and teachers access grade analytics.
- **Skills Practiced:** Create polymorphic Question types, randomize question order, implement scoring logic, generate reports.
- **Key Concepts:** Inheritance, polymorphism, testability with JUnit, object aggregation

4. Bug Tracker / To-Do Task Manager

- **Scenario:** Manage tasks or bugs with priorities, statuses, and deadlines.
- **Skills Practiced:** Use case + class diagrams, Task filtering/sorting, CRUD operations
- **Key Concepts:** Design patterns (Factory or Strategy)

5. Inventory Management System

- **Scenario:** Track stock levels in a store and handle product updates.
- **Skills Practiced:** Product, InventoryItem, Transaction classes, Restocking rules
- **Key Concepts:** Encapsulation, collection operations

6. Personal Budget Tracker

- **Scenario:** Track expenses by category and generate a summary.
- **Skills Practiced:** Category-based sum aggregation, Monthly report generation
- **Key Concepts:** modular design

7. Recipe Manager

- **Scenario:** Users can store, search, and organize cooking recipes.
- **Skills Practiced:** String filtering/search, Tag-based categorization
- **Key Concepts:** Inheritance, console UI

8. Event Ticket Booking System

- **Scenario:** Users browse events, reserve seats, purchase tickets, and cancel reservations. The system prevents double-booking and tracks seat availability.
- **Skills Practiced:** Implement Event, Seat, Reservation, and Customer classes, manage seat allocation, validate booking rules, generate booking summaries.
- **Key Concepts:** Encapsulation, collections, Factory pattern, object relationships, input validation.